



UNIVERSIDAD DE GRANADA

MusiHub

Nombre del estudiante: ALFONSO FRÍAS FUNES

Tutor/a: Rocío Celeste Romero Zaliz

Grado: GRADO EN INGENIERÍA INFORMÁTICA

Universidad: UNIVERSIDAD DE GRANADA

Departamento: Ciencias de la Computación e Inteligencia Artificial

Curso académico: 2025–2026

TRABAJO FIN DE GRADO

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	1
1.3. Objetivo del proyecto	2
1.4. Estructura de la memoria	2
2. Estado del arte	4
2.1. Trabajos relacionados	4
2.1.1. Tabla comparativa de plataformas clave	4
2.1.2. Plataformas comparables	6
2.1.3. Síntesis del análisis e implicaciones para el diseño	6
3. Metodología y planificación	8
3.1. Enfoque metodológico y planificación	8
3.1.1. Trazabilidad entre análisis y planificación	8
4. Análisis y diseño del sistema	10
4.1. Historias de usuario del MVP	10
4.1.1. Tarjetas de historias de usuario	11
4.2. Sistema de alertas y <i>matching</i>	13
4.2.1. Objetivo y requisitos de diseño	13
4.2.2. Modelo de <i>matching</i>	14
4.2.3. Generación, entrega y trazabilidad	14
4.3. Arquitectura técnica del sistema	14
4.3.1. Arquitectura general	14
4.3.2. Cliente móvil (Android)	15
4.3.3. Backend (API)	16
4.3.4. Base de datos	17
4.3.5. Síntesis de decisiones técnicas	18

A. Encuesta a potenciales usuarios	19
A.1. Contexto de la encuesta	19
A.2. Capturas del formulario y resultados	19

Capítulo 1

Introducción

1.1. Contexto

La comunidad musical actual se apoya cada vez más en entornos digitales para conectar profesionales, difundir proyectos y acceder a oportunidades. En este ecosistema participan perfiles diversos: músicos individuales, bandas, docentes, academias, tiendas de música, salas de conciertos, bares con programación, espacios culturales y otros agentes vinculados a la actividad musical. A día de hoy, gran parte de la interacción entre estos actores se produce mediante redes sociales generalistas, grupos de mensajería y plataformas dispersas (anuncios, eventos, compraventa), donde se publican ofertas, se buscan sustituciones, se proponen colaboraciones o se ofrecen clases y material.

Sin embargo, esta interacción suele estar fragmentada: la información aparece repartida entre múltiples canales, con formatos poco estructurados y con baja capacidad de filtrado. Esto dificulta tareas habituales como: encontrar músicos compatibles para un proyecto, localizar oportunidades laborales puntuales (bolos, sustituciones, clases), organizar propuestas de colaboración o identificar rápidamente material musical en venta dentro de un entorno de confianza.

1.2. Motivación

Aunque existen herramientas digitales para comunicarse y promocionarse, falta una solución especializada que centralice y estructure las oportunidades de la comunidad musical en un único espacio. Las redes sociales generalistas facilitan visibilidad, pero no están diseñadas para hacer *matching* entre necesidades (por ejemplo, una sala que necesita un guitarrista para una fecha concreta) y perfiles (músicos con instrumentos, disponibilidad y ubicación definidas). Del mismo modo, las plataformas genéricas de empleo o compraventa no capturan bien la lógica del sector musical

(sustituciones de última hora, formación de bandas, colaboraciones creativas, clases por especialidad, etc.).

Este TFG se motiva por la necesidad de profesionalizar y agilizar la conexión entre actores musicales mediante una aplicación móvil orientada a: (1) estructurar perfiles (instrumentos, nivel, disponibilidad, ubicación, servicios ofrecidos), (2) facilitar publicaciones (eventos, ofertas, clases, colaboraciones, compraventa) y (3) mejorar la relevancia de la información para cada usuario.

Como elemento diferencial, se propone un sistema de alertas personalizadas, inspirado en el modelo de plataformas de empleo tipo InfoJobs, pero adaptado al contexto musical: el usuario recibe notificaciones cuando aparece una oportunidad coherente con su perfil e intereses (por ejemplo, “batería disponible para banda”, “clases de piano en tu zona”, “sustitución para bolo este fin de semana” o “venta de instrumento del tipo que buscas”).

1.3. Objetivo del proyecto

Objetivo General (OG). Diseñar y desarrollar una aplicación móvil orientada a la comunidad musical que facilite la interacción entre músicos, bandas y entidades del sector, centralizando oportunidades (eventos, clases, ofertas, colaboraciones y compraventa) e incorporando un sistema de alertas personalizadas en función del perfil e intereses del usuario.

1.4. Estructura de la memoria

Antes de entrar en detalle en el desarrollo, se establece la estructura de la memoria de forma coherente con un proyecto de software y consensuada con el tutor/a, para mantener un hilo lógico entre análisis, diseño e implementación. En este tipo de trabajos resulta especialmente útil abordar primero (1) el estado del arte y (2) la metodología, ya que condicionan el alcance, las decisiones técnicas y la planificación del proyecto.

De forma resumida, la memoria se organiza de la siguiente manera:

- **Capítulo 1. Introducción:** contextualiza el dominio, justifica la necesidad del proyecto y define el objetivo general.
- **Capítulo 2. Estado del arte:** análisis de aplicaciones y plataformas comparables dentro del ámbito musical y de modelos tipo empleo/bolsa de oportunidades, recogiendo funcionalidades, ventajas, limitaciones y oportunidades de diferenciación.

- **Capítulo 3. Metodología y planificación:** justificación del ciclo de vida seleccionado (cascada vs. ágil) y descripción del enfoque de trabajo. En este proyecto se parte de un enfoque ágil (Scrum) para iterar en sprints y entregar incrementos funcionales.
- **Capítulo 4. Análisis y diseño del sistema:** requisitos, roles de usuario, arquitectura, modelo de datos y diseño de interfaces.
- **Capítulo 5. Implementación:** desarrollo de la app (funcionalidades principales) y decisiones técnicas relevantes.
- **Capítulo 6. Pruebas y validación:** estrategia de pruebas, resultados y verificación del cumplimiento del objetivo general.
- **Capítulo 7. Conclusiones y trabajo futuro:** conclusiones, limitaciones y posibles mejoras/ampliaciones.

Capítulo 2

Estado del arte

2.1. Trabajos relacionados

2.1.1. Tabla comparativa de plataformas clave

Para acotar el análisis y evitar un catálogo excesivo de herramientas, se seleccionan cinco plataformas representativas que cubren patrones funcionales próximos al proyecto: *networking* entre músicos, directorio con filtros, contratación de actuaciones, clasificados multi-categoría y tablón de oportunidades. Como se observa en la Tabla 2.1, todas ellas resuelven una parte del problema, pero ninguna integra de forma completa perfiles musicales estructurados, publicaciones tipificadas y alertas personalizadas con trazabilidad.

Cuadro 2.1: Comparativa de plataformas clave relacionadas con la comunidad musical y las oportunidades.

Nombre	Tipo	País/alcance	Enfoque	Funcionalidades clave	Matching/alertas	Modelo	Pros	Contras	Link
Vampr	App	Global	Banda / networking	Perfiles de músicos Descubrimiento Chat	Sí: matching tipo swipe + notifs de mensajes	Freemium (suscripción)	Comunidad grande Muy social	Funciones pro de pago Menos foco en jobs	Vampr
BandMix	Web	Global (con .es)	Banda / miembros	Directorio por instrumento/estilo Perfil con media Mensajería	Parcial: avisos/email + mensajería	Freemium (premium para contactar)	Filtros potentes Mucha oferta	Contacto suele requerir pago UX web clásica	BandMix
Gigstarter	Web	Europa	Bolos / contratación	Perfiles de artistas Solicitudes de clientes Gestión de reservas	Sí: avisos + recordatorios de eventos	Freemium (plan PRO)	Sin comisiones (según plataforma) Bolos reales	Más cliente- >artista que “jobs” Menos comunidad	Gigstarter
Shutapp	Web	España	Jobs + clases + servicios	Clasificados por categorías Geolocalización/radio Perfiles + anuncios	Parcial: feed/últimos anuncios; sin matching automático	Freemium (destacados de pago)	Multi-categoría (similar al TFG) Publicar fácil	Sin app Depende de masa crítica local	Shutapp
EmpleoParaMúsicos	Web	España	Jobs (tablón)	Agrega ofertas/audiciones Categorías por instrumento Publicar ofertas	No: sin perfiles/matching; consulta manual	Gratis/donaciones (no confirmado)	Radar útil de empleo Mucha variedad	No gestiona candidaturas Sin alertas personalizadas	EmpleoParaMúsicos

2.1.2. Plataformas comparables

A partir de la Tabla 2.1, se analizan las cinco plataformas seleccionadas para identificar por qué resultan comparables, qué parte del problema validan y por qué no cubren por completo la propuesta del proyecto.

Vampr (app, alcance global). Es comparable porque trabaja directamente sobre perfiles de músicos y descubrimiento entre usuarios, dos elementos que también son relevantes en este TFG. Valida que existe interés por conectar personas a partir de afinidades musicales y actividad en la plataforma. Sin embargo, su foco principal está en el *networking* y la interacción social rápida, no en la publicación estructurada de oportunidades ni en un sistema de alertas orientado a ofertas concretas.

BandMix (web, alcance global). Resulta comparable porque combina perfiles musicales detallados con filtros por instrumento, estilo y ubicación, que son criterios centrales para reducir fricción en la búsqueda. Demuestra que el perfil rico y la consulta filtrada aportan valor real al usuario. Aun así, su dinámica sigue siendo mayoritariamente reactiva: el usuario debe buscar y contactar manualmente, y el flujo de conversión queda condicionado por barreras de acceso al contacto.

Gigstarter (web, Europa). Es relevante por acercarse al caso de contratación de actuaciones, mostrando cómo se puede organizar un flujo entre oferta, solicitud y reserva. Valida la necesidad de estructurar oportunidades y de ofrecer cierta gestión alrededor de ellas. La diferencia principal es que su orientación está más centrada en contratación puntual artista-cliente que en una comunidad multirol con anuncios diversos y alertas por encaje.

Shutapp (web, España). Se aproxima al proyecto porque reúne distintas categorías en un único espacio, lo que la convierte en una referencia clara para el concepto de tablón musical unificado. Confirma que tiene sentido concentrar clases, colaboraciones, servicios y anuncios en una misma plataforma. Su principal limitación es la ausencia de *matching* automático, por lo que el descubrimiento de oportunidades sigue dependiendo casi por completo de la búsqueda manual.

EmpleoParaMúsicos (web, España). Es comparable porque actúa como radar de oportunidades y audiciones, cubriendo la necesidad de centralizar información dispersa. Valida la utilidad de un punto único de consulta para músicos que buscan oportunidades. No obstante, no incorpora perfiles completos, *matching* ni gestión de candidaturas o contacto estructurado, de modo que la consulta sigue siendo esencialmente manual.

2.1.3. Síntesis del análisis e implicaciones para el diseño

En conjunto, el análisis muestra un patrón bastante consistente: las plataformas que mejor funcionan incorporan perfiles musicales, algún tipo de filtro por criterios

relevantes y un mecanismo claro de contacto. Sin embargo, también se repiten tres carencias importantes: la fragmentación del ecosistema en soluciones parciales, la ausencia de recomendación proactiva y el ruido generado por anuncios incompletos o poco mantenidos. En otras palabras, el usuario suele tener que repartir su búsqueda entre varios servicios y asumir un esfuerzo manual elevado para encontrar oportunidades útiles.

Estas conclusiones se ven reforzadas por la encuesta recogida en el Anexo A. En ella, los filtros considerados más importantes fueron ciudad/zona (84,8%), instrumento (78,8%) y estilo musical (66,7%), lo que respalda la necesidad de un sistema de búsqueda estructurado. Además, la información considerada obligatoria en un anuncio se concentra en campos como ciudad/zona (87,9%), descripción (81,8%) y fecha y hora (72,7%), lo que refuerza la decisión de exigir publicaciones tipificadas y con datos mínimos obligatorios. Del mismo modo, la preferencia por alertas inmediatas o configurables y la preocupación por el spam o las estafas apoyan la incorporación de un sistema de avisos con control de frecuencia y trazabilidad de envíos.

Por tanto, el proyecto no se plantea como una copia de una plataforma existente, sino como una síntesis de lo que sí funciona y una respuesta a lo que sigue quedando descubierto. A partir de este estado del arte se justifican las decisiones principales del diseño del sistema: perfiles musicales estructurados, anuncios clasificados por tipo, filtros mínimos por criterios musicales y geográficos, y un mecanismo de alertas proactivas explicables mediante reglas y puntuación. Estas decisiones se concretan posteriormente en los requisitos y en la arquitectura del Capítulo 4.

Capítulo 3

Metodología y planificación

3.1. Enfoque metodológico y planificación

El desarrollo del proyecto se ha organizado mediante un enfoque ágil inspirado en Scrum y adaptado al contexto real del TFG. En lugar de aplicar de forma estricta todos los artefactos y ceremonias de Scrum, se ha trabajado con iteraciones quincenales, revisión periódica del avance y definición de objetivos concretos para el siguiente tramo de trabajo. Esta adaptación resulta coherente con un proyecto desarrollado por un único estudiante bajo supervisión académica, donde prima la capacidad de revisar, corregir y reorientar el trabajo de forma incremental.

Cada sprint tiene una duración aproximada de quince días. Al final de cada periodo se mantiene una reunión con la tutora en la que se revisa el trabajo realizado, se comentan problemas detectados y se fijan los objetivos del sprint siguiente. Este esquema permite introducir correcciones de forma continua y mantener alineado el desarrollo del documento y del sistema con los criterios académicos del proyecto.

Como soporte del seguimiento se utiliza un repositorio en GitHub. En él se versiona la documentación actualizada del proyecto y se almacena un documento de sprint por iteración. Dicho documento recoge, de forma resumida, los objetivos previstos, los puntos tratados en la reunión de seguimiento, el trabajo realizado durante el sprint y los objetivos acordados para el siguiente.

3.1.1. Trazabilidad entre análisis y planificación

La planificación seguida parte de un criterio acumulativo: primero se delimita el problema, después se analiza el dominio, a continuación se concretan requisitos y decisiones de diseño, y finalmente se refactoriza la memoria para mejorar coherencia y defensa académica. De este modo, cada sprint no se entiende como una fase cerrada y aislada, sino como una iteración que consolida lo anterior y prepara el siguiente nivel de detalle.

En este proyecto, la trazabilidad entre requisitos y planificación no se ha gestionado mediante un backlog formal, sino mediante la revisión quincenal de prioridades en función del estado del documento y de las decisiones tomadas en tutoría. Así, las conclusiones del estado del arte han servido para orientar los requisitos del sistema; los requisitos, a su vez, han condicionado la arquitectura y las decisiones tecnológicas; y la revisión de capítulos en cada sprint ha permitido corregir redundancias, ajustar el alcance del MVP y mejorar la justificación del diseño.

Este planteamiento mantiene el valor central de una metodología ágil: avanzar mediante entregas parciales revisables, con capacidad de adaptación y con una validación continua del trabajo realizado. En consecuencia, la planificación por sprints no solo organiza el calendario del proyecto, sino que actúa como mecanismo de control de calidad sobre la propia memoria y sobre la evolución conceptual del sistema propuesto.

Capítulo 4

Análisis y diseño del sistema

4.1. Historias de usuario del MVP

Una vez delimitado el problema y analizadas las soluciones existentes, el diseño del sistema se concreta a partir de las funcionalidades que formarán parte del MVP. En este proyecto se adopta un enfoque centrado en historias de usuario, ya que permiten expresar de forma más directa qué valor aporta el sistema a cada perfil y sirven como base para justificar después la arquitectura y las decisiones técnicas. De este modo, los requisitos funcionales quedan integrados en las propias historias de usuario, mientras que los detalles de implementación se reservan para capítulos posteriores.

ID	Historia	Propósito
HU-01	Registro e inicio de sesión	Acceder a funcionalidades privadas de la aplicación.
HU-02	Perfil musical por rol	Mostrar información relevante y mejorar el encaje con oportunidades.
HU-03	Publicación y gestión de anuncios	Crear, editar y cerrar oportunidades tipificadas.
HU-04	Consulta, búsqueda y filtrado	Localizar oportunidades ajustadas a necesidades concretas.
HU-05	Preferencias de alertas	Configurar intereses y frecuencia de avisos.
HU-06	Alertas personalizadas y notificaciones	Recibir oportunidades relevantes sin depender de búsquedas continuas.
HU-07	Favoritos o guardados	Recuperar anuncios de interés para su consulta posterior.

HU-08	Contacto y trazabilidad de avisos	Facilitar la conversión y registrar alertas enviadas.
HU-09	Gestión básica de banda	Representar proyectos musicales colectivos dentro de la aplicación.
HU-10	Publicación de anuncios como banda	Permitir que una banda publique oportunidades y búsqueda de miembros.

4.1.1. Tarjetas de historias de usuario

Identificación	HU-01 Registro e inicio de sesión
Descripción	Como usuario, quiero poder registrarme e iniciar sesión para acceder a las funcionalidades privadas de la aplicación.
Criterios de aceptación	El sistema permite registro y acceso autenticado, y restringe las funciones privadas a usuarios con sesión iniciada.

Identificación	HU-02 Perfil musical por rol
Descripción	Como músico, banda o entidad, quiero disponer de un perfil con instrumentos, estilos, nivel, ciudad y disponibilidad básica para mejorar mi visibilidad y el encaje con oportunidades.
Criterios de aceptación	El perfil puede crearse y editarse, el rol del usuario queda identificado y se almacenan los datos musicales y de disponibilidad definidos para el MVP.

Identificación	HU-03 Publicación y gestión de anuncios
Descripción	Como usuario autorizado, quiero publicar anuncios tipificados —clases, bolos o sustituciones, búsqueda de miembros, eventos y compraventa— y poder editarlos o cerrarlos.
Criterios de aceptación	El sistema permite crear anuncios por tipo, exige campos mínimos según la categoría y permite al autor editarlos o cerrarlos.

Identificación	HU-04 Consulta, búsqueda y filtrado
Descripción	Como usuario, quiero navegar por un listado de anuncios y filtrarlos por tipo, ciudad, instrumento, estilo, fecha y precio o remuneración.
Criterios de aceptación	El sistema muestra un listado consultable y permite combinar filtros básicos para reducir ruido en la búsqueda.

Identificación	HU-05 Preferencias de alertas
Descripción	Como usuario, quiero definir qué oportunidades me interesan y con qué frecuencia deseo recibir avisos para controlar el comportamiento del sistema de alertas.
Criterios de aceptación	El usuario puede configurar tipos de oportunidad, criterios de interés y frecuencia de recepción de alertas.

Identificación	HU-06 Alertas personalizadas y notificaciones
Descripción	Como usuario, quiero recibir notificaciones sobre anuncios relevantes calculados mediante reglas de encaje para no depender de búsquedas manuales continuas.
Criterios de aceptación	El sistema genera alertas cuando una oportunidad supera el umbral de relevancia definido y evita envíos duplicados al mismo usuario.

Identificación	HU-07 Favoritos o guardados
Descripción	Como usuario, quiero guardar anuncios de interés para consultarlos posteriormente sin tener que buscarlos de nuevo.
Criterios de aceptación	El usuario puede guardar y recuperar anuncios marcados como favoritos o guardados desde la aplicación.

Identificación	HU-08 Contacto y trazabilidad de avisos
Descripción	Como usuario, quiero disponer de un medio de contacto claro desde el anuncio —por ejemplo, WhatsApp, email o Instagram— y que el sistema registre las alertas enviadas.
Criterios de aceptación	Cada anuncio muestra una vía de contacto accesible y el sistema conserva un registro de las alertas asociadas al usuario y a la oportunidad.

Identificación	HU-09 Gestión básica de banda
Descripción	Como músico, quiero crear una banda o unirme a una ya existente para representar mi proyecto musical dentro de la aplicación.
Criterios de aceptación	El sistema permite crear una banda, vincular músicos mediante invitación aprobada por un administrador y configurar si la pertenencia a la banda se muestra o no en el perfil público del músico.

Identificación	HU-10 Publicación de anuncios como banda
Descripción	Como banda, quiero publicar anuncios como entidad colectiva para buscar miembros o difundir oportunidades musicales.
Criterios de aceptación	Una banda puede publicar anuncios igual que un usuario individual, incluyendo anuncios orientados a incorporar nuevos miembros.

Estas historias resumen el alcance funcional real del MVP y recogen, de forma compacta, las decisiones derivadas del estado del arte y de la encuesta. Dentro de este alcance se incorpora también una gestión básica de bandas, entendida como una ampliación natural del perfil musical individual y del sistema de anuncios, sin llegar todavía a una gestión avanzada de proyectos musicales. La mensajería interna se considera una funcionalidad deseable por su presencia en algunas plataformas comparables, pero se deja fuera del alcance inmediato del MVP para no desplazar funcionalidades más nucleares.

4.2. Sistema de alertas y *matching*

El motor de alertas constituye el elemento diferencial del proyecto y se sitúa conceptualmente después de las historias de usuario porque depende directamente de ellas: necesita perfiles estructurados, anuncios tipificados, preferencias configurables y un canal de notificación hacia el usuario. Sin estos elementos previos, la lógica de recomendación no tendría una base clara sobre la que operar.

4.2.1. Objetivo y requisitos de diseño

El sistema de alertas debe identificar qué anuncios son relevantes para cada usuario en función de su perfil y de sus preferencias, y comunicar esa relevancia mediante notificaciones *push*. Para que esta funcionalidad resulte defendible en el

contexto del TFG, el diseño debe cumplir varios requisitos: explicar por qué se ha generado una alerta, evitar duplicados, respetar la frecuencia elegida por el usuario y mantener un registro verificable de los envíos realizados.

4.2.2. Modelo de *matching*

Se propone un modelo basado en reglas ponderadas y puntuación. Cada anuncio recibe una valoración para cada usuario en función de coincidencias como ciudad, instrumento, estilo, tipo de oportunidad o disponibilidad. A partir de esa puntuación se establece un umbral mínimo para decidir si la oportunidad debe notificarse.

Este enfoque se ha elegido frente a alternativas más complejas basadas en aprendizaje automático porque ofrece tres ventajas claras para el proyecto: es explicable, resulta viable sin un conjunto de datos histórico amplio y facilita la validación académica de los criterios utilizados. En consecuencia, el *matching* se entiende aquí como una lógica de recomendación transparente y ajustada al alcance del MVP.

4.2.3. Generación, entrega y trazabilidad

La generación de alertas se activa cuando se publica un anuncio y se complementa con el respeto a la frecuencia configurada en las preferencias de cada usuario. Para la entrega se utiliza Firebase Cloud Messaging (FCM), mientras que la trazabilidad se garantiza mediante un registro de notificaciones enviadas asociado al usuario, al anuncio y al momento del envío. Este registro permite evitar alertas duplicadas, analizar el comportamiento del sistema y reforzar la confianza del usuario frente a dos preocupaciones detectadas en la encuesta: el exceso de notificaciones y la presencia de contenido poco fiable.

4.3. Arquitectura técnica del sistema

4.3.1. Arquitectura general

- **Cliente (Flutter/Android):** interfaz de usuario, consumo de la API, gestión de sesión y recepción de notificaciones.
- **API (FastAPI):** lógica de negocio, autenticación y roles, validación de datos y *endpoints* para perfiles, anuncios y preferencias de alertas.
- **Base de datos (PostgreSQL):** almacenamiento consistente de usuarios, perfiles, anuncios y registros de notificaciones.
- **Proceso de alertas:** evalúa nuevas publicaciones frente a las preferencias de los usuarios mediante *scoring* y registra los envíos.

- **FCM:** canal de entrega de notificaciones *push* hacia la aplicación.

Esta separación permite desacoplar la experiencia móvil del núcleo de negocio y mantener la trazabilidad del sistema de alertas.

4.3.2. Cliente móvil (Android)

Rol del cliente móvil en el sistema

El cliente móvil es la interfaz principal del sistema y el punto desde el que los distintos perfiles de usuario interactúan con el producto. Su papel dentro de la arquitectura no es únicamente mostrar información, sino materializar las historias de usuario del MVP: autenticación, gestión de perfil, consulta y publicación de anuncios, configuración de alertas, guardado de oportunidades y recepción de notificaciones *push*. Por ello, la tecnología elegida para este bloque debe priorizar productividad, mantenibilidad e integración fiable con Android.

Tecnologías candidatas para el desarrollo de la app

Para implementar el cliente móvil se valoraron tres alternativas habituales en desarrollo móvil: Flutter, Android nativo con Kotlin y React Native. La comparación se realiza atendiendo a criterios de productividad, calidad de integración en Android, mantenimiento y coste de aprendizaje.

Opción A – Flutter (Dart) Flutter permite desarrollar interfaces con alta consistencia visual y un ritmo de implementación elevado, con buen rendimiento y un ecosistema estable para aplicaciones móviles de este alcance.

Opción B – Android nativo (Kotlin + Jetpack) El desarrollo nativo proporciona un control máximo sobre la plataforma Android, aunque suele implicar un mayor tiempo de desarrollo.

Opción C – React Native (JavaScript/TypeScript) React Native ofrece un enfoque multiplataforma apoyado en JavaScript/TypeScript, pero con una integración nativa más dependiente del ecosistema de librerías.

Elección final: Flutter

Tras la comparación, se selecciona Flutter como tecnología principal para el cliente móvil. La decisión se basa en un equilibrio claro entre velocidad de desarrollo y calidad del resultado, especialmente adecuado para una aplicación con varias pantallas, formularios, filtros y un uso intensivo de notificaciones. Además, permite

estructurar el proyecto de forma ordenada, lo que facilita tanto el mantenimiento como la trazabilidad en la documentación del TFG.

Decisiones internas dentro de Flutter

Con el fin de reducir riesgos y mantener coherencia técnica, se adopta una organización modular por funcionalidades y una integración sencilla con autenticación, consumo de API y notificaciones. Los detalles concretos de librerías y paquetes utilizados se describirán en el capítulo de implementación.

4.3.3. Backend (API)

Papel dentro del sistema

El backend es el componente encargado de sostener la lógica de negocio del sistema y de ofrecer una API consumible desde la aplicación móvil. En este proyecto se ejecuta en local mediante Docker y debe asumir funciones como autenticación, autorización, gestión de anuncios, aplicación de permisos, ejecución del *matching*, trazabilidad de alertas y acceso consistente a la base de datos.

Su presencia es necesaria para que el proyecto no se limite a una interfaz cliente, sino que pueda justificarse como un sistema completo desde el punto de vista de la ingeniería del software. Por ello, la tecnología elegida se valora sobre todo por su capacidad para soportar con claridad estos bloques funcionales, facilitar pruebas sobre la lógica y documentar los *endpoints* de forma comprensible.

Tecnologías candidatas

Opción A – FastAPI (Python) Framework moderno para APIs en Python, orientado a rapidez y claridad, con documentación OpenAPI automática y buen encaje con la lógica del proyecto.

Opción B – NestJS (Node.js + TypeScript) Alternativa sólida y estructurada, aunque con mayor complejidad inicial para el alcance previsto.

Opción C – Spring Boot (Java) Solución muy robusta y defendible, pero más pesada de configurar para un MVP local y acotado.

Decisión y justificación final

Tecnología elegida: FastAPI (Python).

El proyecto requiere integrar autenticación, anuncios, filtros, preferencias, motor de alertas y notificaciones sin introducir complejidad innecesaria en el MVP. FastAPI permite avanzar con rapidez, ofrece documentación OpenAPI automática y encaja especialmente bien con una lógica de *matching* basada en reglas y puntuación. Además, su despliegue en local con Docker resulta sencillo y coherente con el alcance del trabajo.

4.3.4. Base de datos

Finalidad y criterios de diseño

La base de datos es el componente que garantiza persistencia, consistencia y capacidad de consulta sobre la información del sistema. Debe almacenar usuarios y roles, perfiles musicales, anuncios, preferencias de alertas, tokens de dispositivos, bandas y relaciones de membresía, además del registro de notificaciones enviadas. Su diseño está condicionado por dos necesidades principales: soportar búsquedas con filtros combinados y mantener trazabilidad suficiente para justificar decisiones como la caducidad de anuncios, el control de duplicados o la evolución del esquema.

En consecuencia, se prioriza un modelo capaz de manejar integridad relacional, consultas estructuradas y evolución controlada mediante migraciones. Además, debe permitir representar de forma razonable atributos musicales, la pertenencia de un músico a una o varias bandas y preservar un histórico útil para el análisis del comportamiento del sistema de alertas.

Tecnologías candidatas

Opción A – PostgreSQL PostgreSQL es una base de datos relacional consolidada, especialmente adecuada cuando se requieren consultas complejas, consistencia transaccional y capacidad de indexación avanzada.

Opción B – MySQL / MariaDB Alternativa relacional madura y ampliamente utilizada, aunque con menos ventajas diferenciales para este caso concreto.

Opción C – Firebase Firestore Base de datos NoSQL gestionada, útil para modelos simples, pero menos adecuada para un sistema con filtros combinados y control relacional explícito.

Elección final: PostgreSQL

Se selecciona PostgreSQL como base de datos principal por su adecuación al tipo de información del proyecto y, especialmente, por su rendimiento en consultas con

filtros múltiples. Además, permite trabajar con un modelo relacional claro, restricciones, relaciones y migraciones, y se integra de forma natural en un despliegue local mediante Docker Compose. El soporte de JSONB aporta flexibilidad controlada para atributos como instrumentos o estilos en una primera versión del sistema.

4.3.5. Síntesis de decisiones técnicas

Se han seleccionado tecnologías que priorizan productividad y claridad arquitectónica, manteniendo un nivel adecuado de rigor ingenieril. Flutter proporciona una base sólida para el cliente móvil, mientras que FastAPI permite exponer una API REST clara y fácilmente documentable. PostgreSQL aporta consistencia y soporte eficiente para consultas con filtros, necesarias en la búsqueda de anuncios y perfiles. El sistema de alertas se implementa como un proceso auxiliar desacoplado, lo que facilita su evaluación y el control de duplicidades mediante el registro de envíos. La entrega de notificaciones se realiza con FCM. Las decisiones de bajo nivel (librerías concretas y configuración detallada) se describen en el capítulo de implementación.

Apéndice A

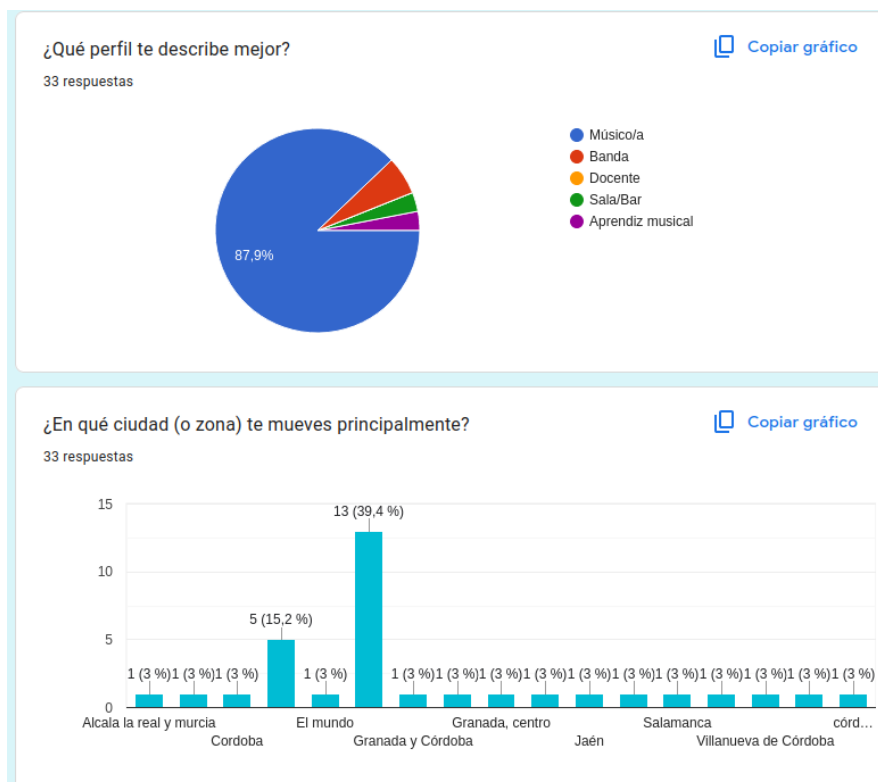
Encuesta a potenciales usuarios

A.1. Contexto de la encuesta

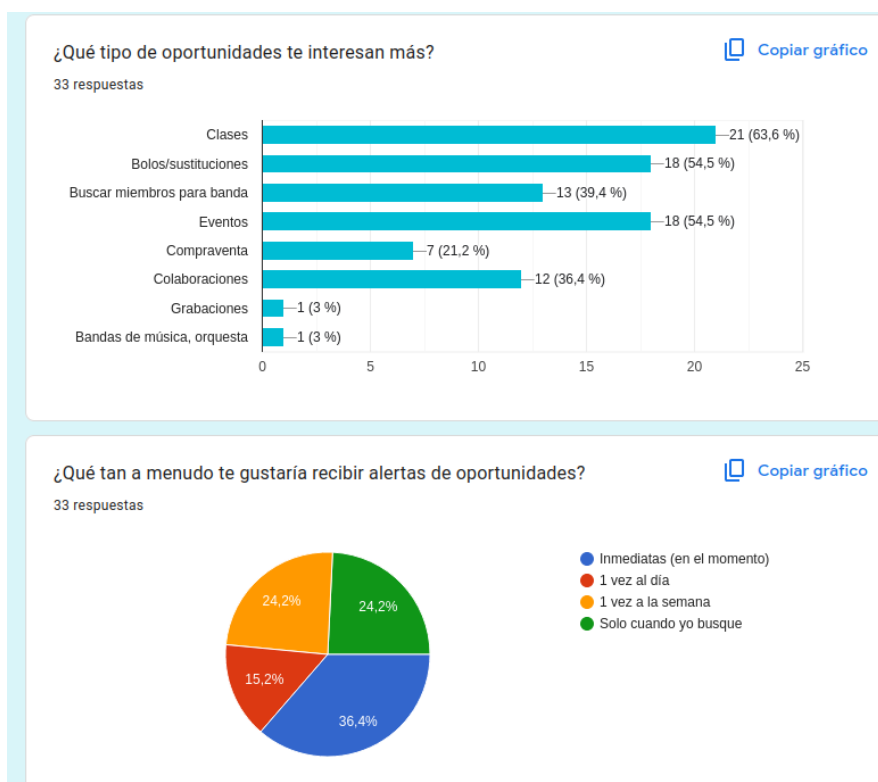
La encuesta se realizó mediante Google Forms durante febrero de 2026 con el objetivo de recoger información exploratoria sobre las preferencias de potenciales usuarios de la aplicación. El formulario se difundió en línea y se obtuvieron 33 respuestas, procedentes principalmente de perfiles vinculados al entorno musical. La participación fue voluntaria y las respuestas se recogieron en remoto, en un único cuestionario estructurado en torno a tipos de oportunidad de interés, filtros de búsqueda, frecuencia de alertas y barreras de uso. Los resultados no se emplean como estudio estadístico representativo, sino como apoyo para justificar decisiones de diseño y priorización del MVP.

A.2. Capturas del formulario y resultados

En las siguientes figuras se muestran las capturas del formulario y del resumen de respuestas exportado desde Google Forms.

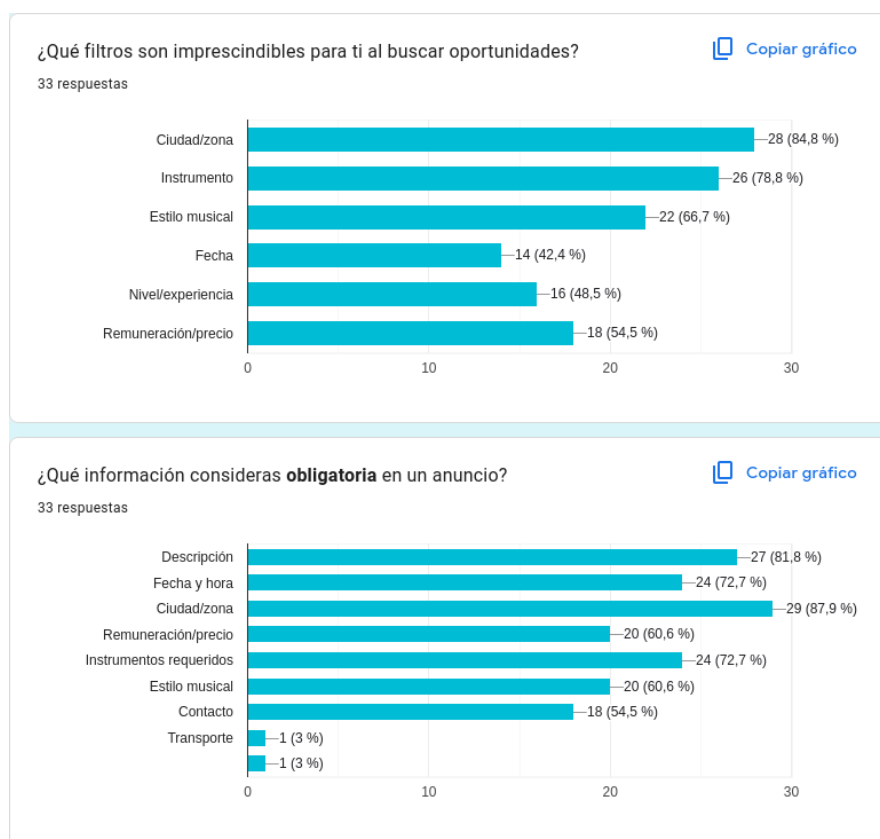


En el perfil de participantes predomina claramente “Músico/a” (87,9 %). En la distribución por zona, la mayor concentración se observa en Granada y Córdoba (39,4 %) y, en segundo lugar, en Córdoba (15,2 %), con el resto de respuestas más dispersas.

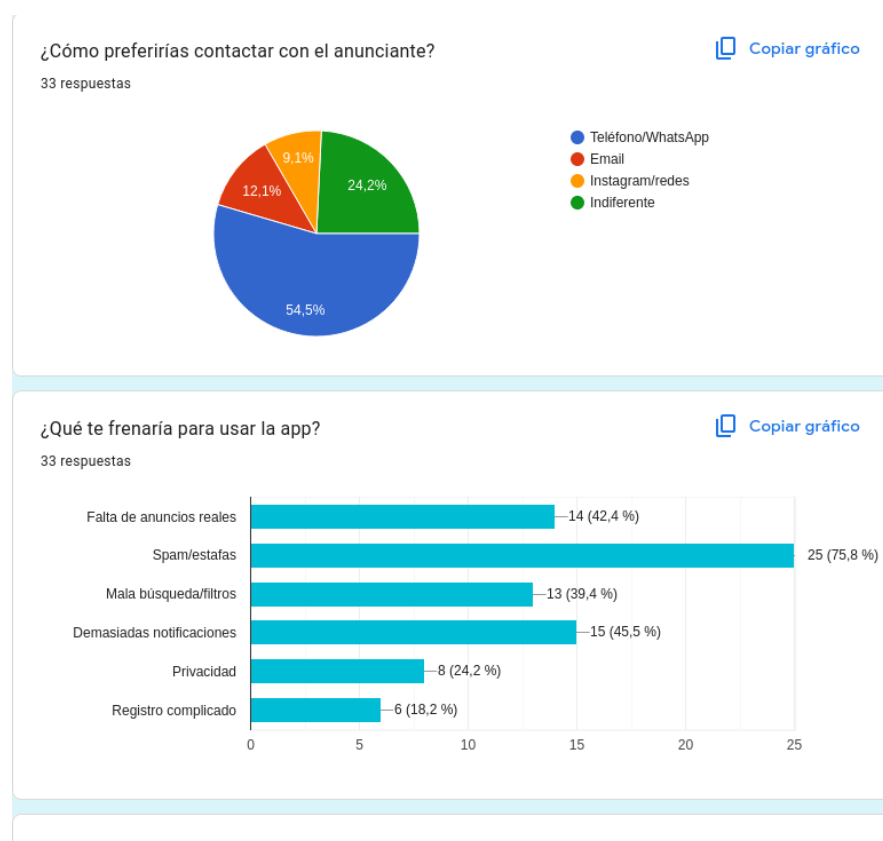


Las oportunidades con mayor interés son clases (63,6 %), bolos/sustituciones (54,5 %)

y eventos (54,5%). Sobre frecuencia de alertas, destaca “inmediatas” (36,4%), seguida de “1 vez a la semana” (24,2%) y “solo cuando yo busque” (24,2%).



En filtros imprescindibles, los más votados son ciudad/zona (84,8%), instrumento (78,8%) y estilo musical (66,7%). En la información obligatoria de un anuncio, lideran ciudad/zona (87,9%), descripción (81,8%), fecha y hora (72,7%) e instrumentos requeridos (72,7%).



El canal de contacto preferido es teléfono/WhatsApp (54,5 %), por encima de email (12,1 %) e Instagram/redes (9,1 %). Como frenos de uso, el principal es spam/estafas (75,8 %), seguido de demasiadas notificaciones (45,5 %) y falta de anuncios reales (42,4 %).

¿Qué función o idea añadirías a esta app que no hayamos mencionado y que te parecería clave?

12 respuestas

- Quizás implementaría también elementos destinados al contacto con técnicos de sonido/visual.
- Ninguna
- Ninguna
- Poder crear tu propio perfil poniendo tu información (instrumento, experiencia, nivel, etc) para poder contactar con otros músicos de la zona deseada
- Búsqueda de salas por estilos
- En los perfiles de los artistas/bandas/profesionales ver donde tienen subidos sus trabajos previos, experiencia, links e incluso formaciones
- Tipo de actividad, profesional o aficionado
- Tocar en la calle

En las respuestas abiertas (12 respuestas) se repiten propuestas como ampliar perfiles (experiencia, nivel, enlaces de trabajo), facilitar búsquedas más específicas por estilo o zona y contemplar necesidades complementarias (p. ej., contacto con técnicos).